

Implementation of Optimal Classification Tree

Basic, warm-starts and tuning

Andrea Rota

Department of Management, Information and Production Engineering
University of Bergamo

April 2025



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

- 1 Introduction
- 2 Optimal Classification Tree
- 3 Warmstart
- 4 Hyperparameter tuning
- 5 Final considerations

Project objective

The goal of this project is to implement one of the classification trees introduced in the paper “Optimal Classification Trees” by Bertsimas and Dunn. Specifically, I have chosen to develop the **Optimal Classification Tree** (OCT) as a Mixed-Integer Optimization (MIO) problem. To improve efficiency, I have incorporated **warm starts** to reduce training times and performed **hyperparameter tuning** to optimize the model’s predictive performance.

Dataset: the Iris dataset

- A well-known dataset in statistical and machine learning research
- Consists of 150 samples, evenly distributed across three species: **Setosa**, **Versicolor**, and **Virginica**.
- The dataset was split into **60% training** and **40% testing** sets to evaluate the model's generalization performance.

	Sepal length (cm)	Sepal width (cm)	Petal length (cm)	Petal width (cm)
Count	150	150	150	150
Mean	5.843	3.054	3.759	1.199
Std	0.828	0.434	1.764	0.763
Min	4.300	2.000	1.000	0.100
Median	5.800	3.000	4.350	1.300
Max	7.900	4.400	6.900	2.500

Table 1: Descriptive statistics of the Iris dataset.

- 1 Introduction
- 2 Optimal Classification Tree**
- 3 Warmstart
- 4 Hyperparameter tuning
- 5 Final considerations

Sets

This table provides an overview of the key sets used in the decision tree model.

Symbol	Description
K	Set of class labels
T_B	Set of branch (internal) nodes in the decision tree
T_L	Set of leaf nodes in the decision tree
$A_R(t)$	Set of right-branch ancestors of node t
$A_L(t)$	Set of left-branch ancestors of node t

Table 2: Notation for sets used in the decision tree model.

Constants

This table summarizes the key constants used in the decision tree formulation.

Symbol	Description
$\hat{L}$	Baseline prediction accuracy, obtained by always predicting the most frequent class
α	Regularization parameter in the objective function, controlling tree complexity
$N_{\min}$	Minimum number of instances required in each leaf node
ϵ_f	Smallest nonzero separation value for feature f , ensuring numerical stability

Table 3: Notation for constants used in the decision tree model.

Decision variables I

The decision variables for the OCT formulation are listed below:

- $a_{j,t}$: Binary variable indicating whether feature j is used for splitting at branch node t ($a_{j,t}^T x_i < b_t$).
- b_t : Real variable representing the split threshold at branch node t .
- $c_{k,t}$: Binary variable indicating if class k is assigned to leaf node t .
- d_t : Binary variable indicating whether branch node t is active.
- l_t : Binary variable indicating whether leaf node t is active.
- L_t : Non-negative integer variable representing the number of misclassified instances at leaf node t .

Decision variables II

- $z_{i,t}$: Binary variable indicating if data point i is assigned to leaf node t .
- N_t : Non-negative integer variable representing the total number of samples in leaf node t .
- $N_{k,t}$: Non-negative integer variable indicating the number of samples in leaf node t belonging to class k .

Constraints: splitting rules

These constraints ensure that split rules are generated only at nodes where a split is decided:

$$\sum_{j=1}^p a_{jt} = d_t, \quad \forall t \in T_B \quad (1)$$

$$0 \leq b_t \leq d_t, \quad \forall t \in T_B \quad (2)$$

If a split is stopped at a branch node t , then no split rule applies to its children:

$$d_t \leq d_{p(t)}, \quad \forall t \in T_B \setminus \{1\} \quad (3)$$

Constraints: label assignment

This constraint guarantees that no label is assigned to empty leaf nodes:

$$\sum_{k=1}^K c_{kt} = l_t, \quad \forall t \in T_L \quad (4)$$

Each data point must be assigned to exactly one leaf node:

$$\sum_{t \in T_L} z_{it} = 1, \quad i = 1, \dots, n \quad (5)$$

Constraints: minimum samples in leaf nodes

Each leaf node, if not empty, must contain at least $N_{\min}$ data points:

$$z_{it} \leq l_t, \quad \forall t \in T_L \quad (6)$$

$$\sum_{i=1}^n z_{it} \geq N_{\min} l_t, \quad \forall t \in T_L \quad (7)$$

Constraints: data routing

Each data point i follows a unique path from root to leaf:

$$a_m^T x_i \geq b_t - (1 - z_{it}),$$
$$i = 1, \dots, n, \quad \forall t \in T_B, \quad \forall m \in A_R(t) \quad (8)$$

$$a_m^T (x_i + \epsilon) \leq b_t + (1 + \epsilon_{\max})(1 - z_{it}),$$
$$i = 1, \dots, n, \quad \forall t \in T_B, \quad \forall m \in A_L(t) \quad (9)$$

Constraints: misclassification calculation

The number of misclassified data points in each leaf node is computed as:

$$N_{kt} = \frac{1}{2} \sum_{i=1}^n (1 + Y_{ik}) z_{it}, \quad k = 1, \dots, K, \quad \forall t \in T_L \quad (10)$$

$$N_t = \sum_{i=1}^n z_{it}, \quad \forall t \in T_L \quad (11)$$

$$L_t \geq N_t - N_{kt} - n(1 - c_{kt}), \quad k = 1, \dots, K, \quad \forall t \in T_L \quad (12)$$

$$L_t \leq N_t - N_{kt} + nc_{kt}, \quad k = 1, \dots, K, \quad \forall t \in T_L \quad (13)$$

$$L_t \geq 0, \quad \forall t \in T_L \quad (14)$$

Constraints: variable domains

Finally, constraints specify the ranges and types of decision variables:

$$z_{it}, l_t \in \{0, 1\}, \quad i = 1, \dots, n, \quad \forall t \in T_L \quad (15)$$

$$s_{jt}, d_t \in \{0, 1\}, \quad j = 1, \dots, p, \quad \forall t \in T_B \quad (16)$$

Objective function

The objective function for the OCT, formulated as a Mixed Integer Optimization problem, aims to minimize the weighted sum of misclassified data points and branch nodes with applied split rules:

$$\min \quad \frac{1}{\hat{L}} \sum_{t \in T_L} L_t + \alpha \sum_{t \in T_B} d_t$$

Where:

- T_L : Set of leaf nodes.
- L_t : Number of misclassified instances for leaf node t .
- T_B : Set of branch nodes.
- d_t : Binary decision variable indicating whether a split rule is applied at branch node t (1 if applied, 0 otherwise).
- α : Regularization parameter that balances the trade-off between minimizing classification error and penalizing model complexity.

Implementation and performance I

The implementation was conducted using Python 3.13 with the Pyomo framework, while Gurobi was selected as the solver.

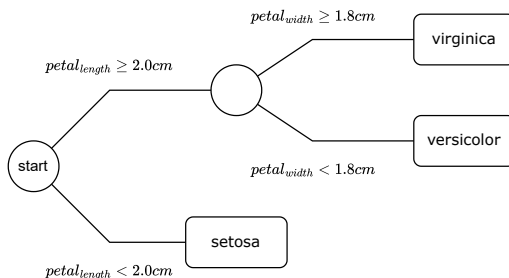


Figure 1: Decision tree with max depth = 2, $\alpha = 0.1$, min samples per leaf = 0.

Implementation and performance II

Act/Pred	Setosa	Versicolor	Virginica
Setosa	29	0	0
Versicolor	0	29	1
Virginica	0	3	28

Table 4: Correlation matrix of predictions.

Metric	Value
Duration	2.11 s
Training accuracy	95.56%
Testing accuracy	96.67%

Table 5: Performance metrics of the model.

As the decision tree depth increases, training times significantly rise: for a depth of 3, the time reaches 150 seconds even with a small dataset. Additionally, training time increases notably with larger datasets in terms of features and instances.

- 1 Introduction
- 2 Optimal Classification Tree
- 3 Warmstart**
- 4 Hyperparameter tuning
- 5 Final considerations

Improving model resolution times

To enhance model resolution times, we leverage CART to generate effective warm start solutions for the MIO problem:

- Initial feasible solutions for MIO can be derived from **CART splits** (a_{tf} , b_t , and d_t), inferring the values of remaining variables.
- Parameters $N_{\min}$ and α remain consistent between CART and MIO.
- Furthermore, MIO solutions for a given depth D can be utilized as warm starts for solving the problem at depth $D + 1$, leading to a significant **reduction in training times**.

Now, we are prepared to perform hyperparameter tuning.

- ① Introduction
- ② Optimal Classification Tree
- ③ Warmstart
- ④ Hyperparameter tuning
- ⑤ Final considerations

Which parameters do we tune?

- Maximal depth $D_{\max}$: Incremental tree building from depth 1 to $D_{\max}$.
- Complexity parameter α :
 - The objective function is defined as:

$$\min \frac{1}{\hat{L}} \sum_{t \in \mathcal{T}_L} L_t + \alpha \sum_{t \in \mathcal{T}_B} d_t$$

- To efficiently tune α :
 - Discretize and evaluate multiple values to minimize the number of MIO problems solved.
 - Reformulate the penalty term as a constraint:

$$\min \frac{1}{\hat{L}} \sum_{t \in \mathcal{T}_L} L_t \quad \text{s.t.} \quad \sum_{t \in \mathcal{T}_B} d_t \leq C$$

- Search for integral values of C , where $C_{\max} = 2^D - 1$ indicates the maximum allowable splits.

Post-processing solutions

- Remove solutions that are not optimal for any value of α .
- Evaluate each solution on a validation set to identify the best-performing option.

Algorithm steps

- ① Set $D_{\max}$ and $N_{\min}$.
- ② For $D = 1, \dots, D_{\max}$:
 - ① For $C = 1, \dots, 2^D - 1$:
 - ① Run CART with $\alpha = 0$, trim to depth D and C splits.
 - ② Choose the best warm start from the pool.
 - ③ Solve MIO problem for depth D and C splits.
 - ④ Add MIO solution to the warm start pool.
- ③ Post-process the solution pool.
- ④ Identify the best solution on a validation set.

Results I

Here we see the results of the first part of the tuning process.

Model type	D	C	Time (s)	Train acc.	Warm-start
CART	2	1	0.0005	66.67%	-
MIO	2	1	0.1516	66.67%	CART
CART	2	2	0.0006	95.56%	-
MIO	2	2	1.7122	91.11%	CART
CART	2	3	0.0007	95.56%	-
MIO	2	3	7.1410	95.56%	CART
CART	3	1	0.0007	66.67%	-
MIO	3	1	0.1743	63.33%	MIO(2-3)
CART	3	2	0.0007	95.56%	-
MIO	3	2	2.0073	95.56%	MIO(2-3)
CART	3	3	0.0008	96.67%	-
MIO	3	3	35.8626	96.67%	CART
CART	3	4	0.0006	96.67%	-

Results II

Model type	D	C	Time (s)	Train acc.	Warm-start
MIO	3	4	124.6750	97.78%	MIO(3-3)
CART	3	5	0.0014	96.67%	-
MIO	3	5	82.8625	98.89%	MIO(3-4)
CART	3	6	0.0011	96.67%	-
MIO	3	6	101.0794	98.89%	MIO(3-5)
CART	3	7	0.0011	96.67%	-
MIO	3	7	46.5915	98.89%	MIO(3-4)

Results III

Post-processing the solution pool, now only MIO models are included.

Depth	α	Time (s)	Train acc.	Test acc.
2	1	5.33	90.00%	86.67%
2	2	3.79	91.11%	91.67%
2	3	3.02	95.56%	96.67%
3	1	136.70	98.89%	98.33%
3	2	211.14	98.89%	96.67%
3	3	155.99	95.56%	95.00%
3	4	46.46	95.56%	95.00%
3	5	12.28	95.56%	96.67%
3	6	13.55	90.00%	88.33%
3	7	12.60	90.00%	88.33%

Results IV

The final selected model has $\alpha = 1$ and *Depth* = 3.

Act/Pred	Setosa	Versicolor	Virginica
Setosa	29	0	0
Versicolor	0	29	1
Virginica	0	0	31

Table 6: Correlation matrix of predictions.

Results V

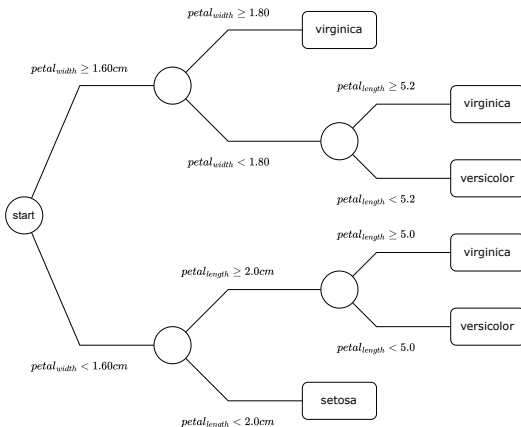


Figure 2: Decision tree with max depth = 3, $\alpha = 1$.

- ① Introduction
- ② Optimal Classification Tree
- ③ Warmstart
- ④ Hyperparameter tuning
- ⑤ Final considerations

Model performance

The performance of the proposed model is comparable to, and in some cases even superior to, traditional CART models when applied to small datasets and shallow trees. However, as model complexity increases, the computational cost may outweigh the benefits in terms of accuracy.

Thank you for your attention!
Any questions?